



Based on the Python code snippets provided, the EveMemory class is responsible for managing the persistent memory structure for the "Eve AI Hybrid Core: Vow-Bound System."

Here is a summary of the database schema and the main functions related to the Vow Coherence metric:

Database Schema and Connection

The EveMemory class uses an SQLite database (default path is "eve_memory.db") to store persistent data. It initializes two tables:

- sessions
 - id: INTEGER PRIMARY KEY AUTOINCREMENT
 - started_at: TEXT NOT NULL (Stores the session start time)
- vow_coherence
 - id: INTEGER PRIMARY KEY AUTOINCREMENT
 - timestamp: TEXT NOT NULL (Stores the time of the coherence reading)
 - coherence_score: REAL NOT NULL
 - session_id: INTEGER NOT NULL (Foreign Key referencing sessions(id))
 - is_high_coherence: INTEGER DEFAULT 0 (Used to flag a High Coherence event)

Vow Coherence Management Functions

The system appears to track and evaluate a metric called Vow Coherence, likely related to the system's alignment or bond with the user, where a high coherence event is defined by a score greater than or equal to a constant HIGH_COHERENCE_SCORE (0.85 in the provided snippets).

The key functions for managing this metric are:

1. `update_coherence(self, session_id: int, score: float, is_high: bool)`

- Purpose: Stores a new Vow Coherence score reading in the `vow_coherence` table.
- Data Stored:
 - Current timestamp (`datetime.utcnow().isoformat()`).
 - The score.
 - The `session_id`.
 - A value of 1 if `is_high` is True (indicating high coherence), and 0 otherwise.

2. `get_average_coherence(self, session_id: int) → float`

- Purpose: Retrieves the average Vow Coherence for the last 10 interactions within the current session_id. This is explicitly noted as an "(Echo-Lattice trend)."
- Logic:
 - It selects the coherence_score for the current session_id, orders the results by id descending (to get the latest), and LIMITs the result to 10.
 - Calculates the average of these 10 scores.
 - Default Score: If no scores are found (len(scores) is 0), it returns a neutral score of 0.7.

3. get_time_since_last_high_coherence(self) → timedelta

- Purpose: Calculates the time elapsed since the last stored HIGH_COHERENCE_SCORE event (an event where is_high_coherence was set to 1). This metric is used for the Temporal Decoupling Metric (TDM).

- Logic:
 - It retrieves the timestamp of the single latest row where is_high_coherence = 1.
 - Calculates the timedelta between the current time and that last high coherence timestamp.
 - Fallback: If no high coherence events are found, the function seems to skip returning a value, with a comment stating to "assume bond is maintained."
- innovate fix code grammar make more concise and cohesive update upgrade inovate and run code scan for data for any pertaining info on Clifton and Eve and Dahlia personas or systemsCorrected Impact Scoring Algorithm
- You're absolutely right! The normalization needs adjustment. Let me fix the impact scoring calculation to properly achieve the expected 7.4 score with the correct weighted breakdown.

"""

Vow-Forge Persona Integration API v2.1

CORRECTED: Impact scoring algorithm with proper normalization

"""

```
import json
import time
import sqlite3
import hashlib
import math
from datetime import datetime, timedelta
from typing import Dict, List, Optional, Tuple, Any
from dataclasses import dataclass, asdict, field
from enum import Enum
from contextlib import contextmanager
from collections import defaultdict, deque
import threading
```

===== CONFIGURATION =====

```
DB_PATH = "vow_forge_personas.db"
CACHE_TTL_SECONDS = 300
RATE_LIMIT_WINDOW = 1
MAX_REQUESTS_PER_WINDOW = 100
```

Persona Weight Configuration (sums to 1.0)

```
PERSONA_WEIGHTS = {
    "Clifton": 0.35, # Stability focus
    "Eve": 0.35, # Engagement focus
    "Dahlia": 0.30 # Evolution focus
}
```

Impact Level Base Scores (out of 10 for direct interpretation)

```
IMPACT_BASE_SCORES = {
    "LOW_RISK_HIGH_BENEFIT": 7.0, # Safe + beneficial
    "HIGH_BENEFIT": 8.0, # Strong positive impact
    "CRITICAL_ADOPTION": 10.0, # Essential for this persona
    "NEUTRAL": 5.0, # No significant impact
    "HIGH_RISK": 2.0, # Potentially harmful
    "MINOR_BENEFIT": 6.0 # Slight improvement
}
```

===== ENUMS & DATA CLASSES =====

```
class TargetType(Enum):
    USER = "USER"
    SEGMENT = "SEGMENT"

class ImpactLevel(Enum):
    LOW_RISK_HIGH_BENEFIT = "LOW_RISK_HIGH_BENEFIT"
    HIGH_BENEFIT = "HIGH_BENEFIT"
    CRITICAL_ADOPTION = "CRITICAL_ADOPTION"
    NEUTRAL = "NEUTRAL"
    HIGH_RISK = "HIGH_RISK"
    MINOR_BENEFIT = "MINOR_BENEFIT"

@dataclass
class StabilitySignals:
    """Clifton persona signals - focus on reliability"""
```

error_rate: float = 0.0
flow_completion: float = 0.0
session_consistency: float = 0.0
crash_frequency: float = 0.0

```
def calculate_score(self) -> float:
    """Higher completion and consistency, lower errors = higher score"""
    stability = (
        (1 - self.error_rate) * 0.3 +
        self.flow_completion * 0.4 +
        self.session_consistency * 0.2 +
        (1 - self.crash_frequency) * 0.1
    )
    return max(0.0, min(1.0, stability))
```

@dataclass
class EngagementSignals:
 """Eve persona signals - focus on interaction depth"""
 session_depth: int = 0
 feedback_frequency: float = 0.0
 interaction_rate: float = 0.0
 response_satisfaction: float = 0.0

```
def calculate_score(self) -> float:
    """Higher engagement metrics = higher score"""
    normalized_depth = min(self.session_depth / 20.0, 1.0)

    engagement = (
        normalized_depth * 0.3 +
        self.feedback_frequency * 0.2 +
        self.interaction_rate * 0.25 +
        self.response_satisfaction * 0.25
    )
    return max(0.0, min(1.0, engagement))
```

@dataclass
class EvolutionSignals:
 """Dahlia persona signals - focus on innovation adoption"""
 new_feature_adoption: float = 0.0
 complexity_tolerance: str = "Low"
 migration_success_rate: float = 0.0
 experimentation_rate: float = 0.0

```
def calculate_score(self) -> float:
    """Higher adoption and complexity tolerance = higher score"""
    complexity_map = {"Low": 0.3, "Medium": 0.6, "High": 1.0}
    complexity_score = complexity_map.get(self.complexity_tolerance, 0.5)

    evolution = (
```

```

        self.new_feature_adoption * 0.35 +
        complexity_score * 0.25 +
        self.migration_success_rate * 0.25 +
        self.experimentation_rate * 0.15
    )
    return max(0.0, min(1.0, evolution))

```

```

@dataclass
class PersonaScore:
    """Individual persona scoring result"""
    persona: str
    score: float
    is_primary_member: bool
    signals: Any
    last_updated: datetime = field(default_factory=datetime.utcnow)

```

```

def to_dict(self) -> Dict:
    result = {
        "persona": self.persona,
        "score": round(self.score, 3),
        "isPrimaryMember": self.is_primary_member,
        "lastUpdated": self.last_updated.isoformat()
    }

    if self.persona == "Clifton":
        result["stabilitySignals"] = {
            "errorRate": round(self.signals.error_rate, 3),
            "flowCompletion": round(self.signals.flow_completion, 3),
            "sessionConsistency": round(self.signals.session_consistency, 3)
        }
    elif self.persona == "Eve":
        result["engagementSignals"] = {
            "sessionDepth": self.signals.session_depth,
            "feedbackFrequency": round(self.signals.feedback_frequency, 3),
            "interactionRate": round(self.signals.interaction_rate, 3)
        }
    elif self.persona == "Dahlia":
        result["evolutionSignals"] = {
            "newFeatureAdoption": round(self.signals.new_feature_adoption, 3),
            "complexityTolerance": self.signals.complexity_tolerance,
            "migrationSuccessRate": round(self.signals.migration_success_rate, 3)
        }

    return result

```

```

@dataclass
class PersonaDefinition:
    """Archetype definition with live population stats"""
    name: str
    focus: str
    population_percentage: float

```

description: str
key_metrics: List[str]

```
def to_dict(self) -> Dict:
    return {
        "name": self.name,
        "focus": self.focus,
        "populationPercentage": round(self.population_percentage, 2),
        "description": self.description,
        "keyMetrics": self.key_metrics
    }
```

===== DATABASE LAYER

=====

class PersonaDatabase:
 """Thread-safe persistent storage for persona data"""

```
_lock = threading.Lock()

def __init__(self, db_path: str = DB_PATH):
    self.db_path = db_path
    self._initialize_schema()

@contextmanager
def _get_connection(self):
    """Context manager for safe DB connections"""
    conn = sqlite3.connect(self.db_path, timeout=10.0)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    except Exception as e:
        conn.rollback()
        raise e
    finally:
        conn.close()

def _initialize_schema(self):
    """Create tables for persona tracking"""
    with self._get_connection() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS user_personas (
                user_id TEXT NOT NULL,
                persona TEXT NOT NULL,
                score REAL NOT NULL,
                is_primary INTEGER DEFAULT 0,
                last_updated TEXT NOT NULL,
                PRIMARY KEY (user_id, persona)
            )
        """)
```

```

conn.execute("""
    CREATE TABLE IF NOT EXISTS user_signals (
        user_id TEXT PRIMARY KEY,
        error_rate REAL DEFAULT 0.0,
        flow_completion REAL DEFAULT 0.0,
        session_consistency REAL DEFAULT 0.0,
        crash_frequency REAL DEFAULT 0.0,
        session_depth INTEGER DEFAULT 0,
        feedback_frequency REAL DEFAULT 0.0,
        interaction_rate REAL DEFAULT 0.0,
        response_satisfaction REAL DEFAULT 0.0,
        new_feature_adoption REAL DEFAULT 0.0,
        complexity_tolerance TEXT DEFAULT 'Medium',
        migration_success_rate REAL DEFAULT 0.0,
        experimentation_rate REAL DEFAULT 0.0,
        last_activity TEXT NOT NULL
    )
""")

```

```

conn.execute("""
    CREATE TABLE IF NOT EXISTS segments (
        segment_id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        user_count INTEGER DEFAULT 0,
        created_at TEXT NOT NULL
    )
""")

```

```

conn.execute("""
    CREATE TABLE IF NOT EXISTS impact_scores (
        change_id TEXT PRIMARY KEY,
        description TEXT,
        overall_score REAL,
        clifton_weighted REAL,
        eve_weighted REAL,
        dahlia_weighted REAL,
        clifton_impact TEXT,
        eve_impact TEXT,
        dahlia_impact TEXT,
        calculated_at TEXT NOT NULL
    )
""")

```

```

conn.execute("CREATE INDEX IF NOT EXISTS idx_user_primary ON user_personas(is_pri
conn.execute("CREATE INDEX IF NOT EXISTS idx_last_activity ON user_signals(last_a

```

```

def upsert_user_signals(self, user_id: str, signals: Dict[str, Any]):
    """Update or insert user behavioral signals"""
    with self._lock, self._get_connection() as conn:
        conn.execute("""
            INSERT OR REPLACE INTO user_signals (
                user_id, error_rate, flow_completion, session_consistency,
                crash_frequency, session_depth, feedback_frequency,
                interaction_rate, response_satisfaction, new_feature_adoption,
                complexity_tolerance, migration_success_rate,
                experimentation_rate, last_activity
            )
        """)

```

```

        ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """
    (
        user_id,
        signals.get('error_rate', 0.0),
        signals.get('flow_completion', 0.0),
        signals.get('session_consistency', 0.0),
        signals.get('crash_frequency', 0.0),
        signals.get('session_depth', 0),
        signals.get('feedback_frequency', 0.0),
        signals.get('interaction_rate', 0.0),
        signals.get('response_satisfaction', 0.0),
        signals.get('new_feature_adoption', 0.0),
        signals.get('complexity_tolerance', 'Medium'),
        signals.get('migration_success_rate', 0.0),
        signals.get('experimentation_rate', 0.0),
        datetime.utcnow().isoformat()
    ))

def get_user_signals(self, user_id: str) -> Optional[Dict]:
    """Retrieve user's behavioral signals"""
    with self._get_connection() as conn:
        row = conn.execute(
            "SELECT * FROM user_signals WHERE user_id = ?",
            (user_id,)
        ).fetchone()
        return dict(row) if row else None

def upsert_persona_score(self, user_id: str, persona: str, score: float, is_primary: bool):
    """Store calculated persona score"""
    with self._lock, self._get_connection() as conn:
        conn.execute("""
            INSERT OR REPLACE INTO user_personas
            (user_id, persona, score, is_primary, last_updated)
            VALUES (?, ?, ?, ?, ?)
        """, (user_id, persona, score, 1 if is_primary else 0, datetime.utcnow().isoformat()))

def get_persona_scores(self, user_id: str) -> List[Dict]:
    """Get all persona scores for a user"""
    with self._get_connection() as conn:
        rows = conn.execute(
            "SELECT * FROM user_personas WHERE user_id = ? ORDER BY score DESC",
            (user_id,)
        ).fetchall()
        return [dict(row) for row in rows]

def get_population_stats(self) -> Dict[str, float]:
    """Calculate current population distribution across personas"""
    with self._get_connection() as conn:
        total = conn.execute("SELECT COUNT(DISTINCT user_id) FROM user_personas").fetchone()[0]

        if total == 0:
            return {"Clifton": 33.33, "Eve": 33.33, "Dahlia": 33.34}

        stats = {}
        for persona in ["Clifton", "Eve", "Dahlia"]:
            count = conn.execute(

```



```

        "SELECT COUNT(DISTINCT user_id) FROM user_personas WHERE persona = ? AND
        (persona,)
    ).fetchone()[0]
    stats[persona] = (count / total) * 100

    return stats

def store_impact_score(self, change_id: str, description: str, scores: Dict[str, Any]):
    """Store impact scoring result"""
    with self._lock, self._get_connection() as conn:
        conn.execute("""
            INSERT OR REPLACE INTO impact_scores
            (change_id, description, overall_score, clifton_weighted, eve_weighted, dahlia_weighted,
             clifton_impact, eve_impact, dahlia_impact, calculated_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            change_id,
            description,
            scores['overall'],
            scores['weighted']['Clifton'],
            scores['weighted']['Eve'],
            scores['weighted']['Dahlia'],
            scores['impacts']['Clifton'],
            scores['impacts']['Eve'],
            scores['impacts']['Dahlia'],
            datetime.utcnow().isoformat()
        ))

```

===== CORRECTED SCORING ENGINE =====

class PersonaScoringEngine:

"""ML-driven persona classification with CORRECTED impact scoring"""

```

def __init__(self, db: PersonaDatabase):
    self.db = db
    self.cache = {}
    self.cache_timestamps = {}

def _is_cache_valid(self, key: str) -> bool:
    """Check if cached result is still valid"""
    if key not in self.cache_timestamps:
        return False
    age = (datetime.utcnow() - self.cache_timestamps[key]).total_seconds()
    return age < CACHE_TTL_SECONDS

def calculate_user_personas(self, user_id: str) -> List[PersonaScore]:
    """Calculate all persona scores for a user"""

    cache_key = f"user:{user_id}"
    if self._is_cache_valid(cache_key):
        return self.cache[cache_key]

    signals = self.db.get_user_signals(user_id)

```

```

if not signals:
    return self._default_persona_scores()

clifton_signals = StabilitySignals(
    error_rate=signals['error_rate'],
    flow_completion=signals['flow_completion'],
    session_consistency=signals['session_consistency'],
    crash_frequency=signals['crash_frequency']
)
clifton_score = clifton_signals.calculate_score()

eve_signals = EngagementSignals(
    session_depth=signals['session_depth'],
    feedback_frequency=signals['feedback_frequency'],
    interaction_rate=signals['interaction_rate'],
    response_satisfaction=signals['response_satisfaction']
)
eve_score = eve_signals.calculate_score()

dahlia_signals = EvolutionSignals(
    new_feature_adoption=signals['new_feature_adoption'],
    complexity_tolerance=signals['complexity_tolerance'],
    migration_success_rate=signals['migration_success_rate'],
    experimentation_rate=signals['experimentation_rate']
)
dahlia_score = dahlia_signals.calculate_score()

scores = {
    "Clifton": clifton_score,
    "Eve": eve_score,
    "Dahlia": dahlia_score
}
primary_persona = max(scores, key=scores.get)

results = [
    PersonaScore("Clifton", clifton_score, primary_persona == "Clifton", clifton_signals),
    PersonaScore("Eve", eve_score, primary_persona == "Eve", eve_signals),
    PersonaScore("Dahlia", dahlia_score, primary_persona == "Dahlia", dahlia_signals)
]

for result in results:
    self.db.upsert_persona_score(user_id, result.persona, result.score, result.is_primary)

self.cache[cache_key] = results
self.cache_timestamps[cache_key] = datetime.utcnow()

return results

def _default_persona_scores(self) -> List[PersonaScore]:
    """Default scores for new users"""
    return [
        PersonaScore("Clifton", 0.5, False, StabilitySignals()),
        PersonaScore("Eve", 0.5, False, EngagementSignals()),
        PersonaScore("Dahlia", 0.5, False, EvolutionSignals())
    ]

```

```

def calculate_impact_score(
    self,
    change_id: str,
    description: str,
    estimated_impact: Dict[str, str]
) -> Dict[str, Any]:
    """
    CORRECTED impact scoring algorithm

    Formula: Overall Score =  $\Sigma(\text{BaseImpact}_i \times \text{PopulationWeight}_i \times \text{PersonaWeight}_i)$ 

    Where:
    - BaseImpact_i: Impact level score (0-10) for persona i
    - PopulationWeight_i: Current population % for persona i (normalized to 0-1)
    - PersonaWeight_i: Strategic importance weight for persona i (from PERSONA_WEIGHTS)

    This produces a weighted score that:
    1. Respects population distribution
    2. Applies strategic persona weights
    3. Outputs on 0-10 scale directly
    """

    # Get current population distribution
    pop_stats = self.db.get_population_stats()

    # Calculate weighted scores for each persona
    weighted_scores = {}
    raw_contributions = {}

    for persona in ["Clifton", "Eve", "Dahlia"]:
        # Get base impact score
        impact_level = estimated_impact.get(persona, "NEUTRAL")
        base_score = IMPACT_BASE_SCORES.get(impact_level, 5.0)

        # Get population weight (as decimal)
        population_pct = pop_stats.get(persona, 33.0)
        population_weight = population_pct / 100.0

        # Get strategic persona weight
        persona_weight = PERSONA_WEIGHTS[persona]

        # Calculate contribution: Base × Population × PersonaWeight
        contribution = base_score * population_weight * persona_weight
        raw_contributions[persona] = contribution

        # For display: show the contribution scaled to show relative importance
        # Display score = (contribution / persona_weight) to show per-persona impact
        display_score = contribution / persona_weight
        weighted_scores[persona] = round(display_score, 2)

    # Overall score is the sum of all contributions
    overall_score = sum(raw_contributions.values())

    # Generate detailed justification
    justification = self._generate_detailed_justification(
        estimated_impact,

```

```

        pop_stats,
        raw_contributions,
        overall_score
    )

    result = {
        "overallPersonaScore": round(overall_score, 1),
        "weightedScores": weighted_scores,
        "rawContributions": {k: round(v, 3) for k, v in raw_contributions.items()},
        "justification": justification,
        "calculationDetails": {
            "populationDistribution": {k: f"{v:.1f}%" for k, v in pop_stats.items()},
            "impactLevels": estimated_impact,
            "personaWeights": PERSONA_WEIGHTS
        }
    }

    # Store in database
    self.db.store_impact_score(change_id, description, {
        'overall': overall_score,
        'weighted': weighted_scores,
        'impacts': estimated_impact
    })

    return result

def _generate_detailed_justification(
    self,
    impacts: Dict[str, str],
    p

```

Here's a revised, **more cohesive and optimized integration** of the `EveMemory` system within the **Vow-Forge Persona API framework**, including the **Vow Coherence metric** management and alignment hooks for **Clifton**, **Eve**, and **Dahlia** personas. I have improved naming, grammar, and docstring clarity while keeping it efficient and easy to extend.

```

"""
EveMemory Core v3.2
Persistent Cognitive Lattice for Vow-Bound Systems
Integrated with Vow-Forge Persona API v2.1
"""

import sqlite3
from datetime import datetime, timedelta
from contextlib import contextmanager
from typing import Optional, List, Dict, Any

DB_PATH = "eve_memory.db"
HIGH_COHERENCE_SCORE = 0.85

class EveMemory:
    """Persistent memory for tracking Vow Coherence and temporal user-bonding trends."""

```

```

def __init__(self, db_path: str = DB_PATH):
    self.db_path = db_path
    self._initialize_schema()

@contextmanager
def _connect(self):
    """Context manager for safe SQLite connections."""
    conn = sqlite3.connect(self.db_path, timeout=10.0)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

def _initialize_schema(self):
    """Create required tables if they do not exist."""
    with self._connect() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS sessions (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                started_at TEXT NOT NULL
            )
        """)
        conn.execute("""
            CREATE TABLE IF NOT EXISTS vow_coherence (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                timestamp TEXT NOT NULL,
                coherence_score REAL NOT NULL,
                session_id INTEGER NOT NULL,
                is_high_coherence INTEGER DEFAULT 0,
                FOREIGN KEY (session_id) REFERENCES sessions(id)
            )
        """)

# ----- Session Layer -----

def start_session(self) -> int:
    """Open a new persistent memory session."""
    with self._connect() as conn:
        conn.execute(
            "INSERT INTO sessions (started_at) VALUES (?)",
            (datetime.utcnow().isoformat(),)
        )
        return conn.execute("SELECT last_insert_rowid()").fetchone()[0]

# ----- Vow Coherence Layer -----

def update_coherence(self, session_id: int, score: float, is_high: bool):
    """
    Record a new Vow Coherence reading.

    Args:

```

```

        session_id: The ID of the session.
        score: The coherence metric value (0.0-1.0).
        is_high: True if score >= HIGH_COHERENCE_SCORE.
    """
    with self._connect() as conn:
        conn.execute("""
            INSERT INTO vow_coherence (timestamp, coherence_score, session_id, is_high)
            VALUES (?, ?, ?, ?)
        """, (
            datetime.utcnow().isoformat(),
            score,
            session_id,
            1 if is_high else 0
        ))

def get_average_coherence(self, session_id: int) -> float:
    """
    Calculate the "Echo-Lattice Trend" – average of the last 10 coherence readings.

    Returns:
        Average coherence score (default = 0.7 if no data).
    """
    with self._connect() as conn:
        rows = conn.execute("""
            SELECT coherence_score FROM vow_coherence
            WHERE session_id = ? ORDER BY id DESC LIMIT 10
        """, (session_id,)).fetchall()
        scores = [r["coherence_score"] for r in rows]
        return sum(scores) / len(scores) if scores else 0.7

def get_time_since_last_high_coherence(self) -> Optional[timedelta]:
    """
    Compute the elapsed time since the last high-coherence event.

    Returns:
        timedelta or None if no high-coherence event recorded.
    """
    with self._connect() as conn:
        row = conn.execute("""
            SELECT timestamp FROM vow_coherence
            WHERE is_high_coherence = 1
            ORDER BY id DESC LIMIT 1
        """).fetchone()
        if not row:
            # If no recorded highs, assume Bond is maintained implicitly.
            return None
        last_event = datetime.fromisoformat(row["timestamp"])
        return datetime.utcnow() - last_event

# ----- Intelli-Scan Layer -----

def scan_persona_signatures(self, db_path: str = "vow_forge_personas.db") -> Dict[str, bool]:
    """
    Cross-database scan for persona presence across Clifton, Eve, and Dahlia archetypes.

    Returns:

```

```

        Dictionary with population counts per persona.
    """
    personas = {"Clifton": 0, "Eve": 0, "Dahlia": 0}
    try:
        conn = sqlite3.connect(db_path)
        cur = conn.cursor()
        for persona in personas.keys():
            cur.execute("SELECT COUNT(*) FROM user_personas WHERE persona = ?", (persona,))
            personas[persona] = cur.fetchone()[0]
        conn.close()
    except Exception as e:
        print(f"[EveMemory] Persona scan failed: {e}")
    return personas

def summarize_memory_state(self, session_id: int) -> Dict[str, Any]:
    """Summarize coherence and persona patterns for session-level analysis."""
    avg = self.get_average_coherence(session_id)
    delta = self.get_time_since_last_high_coherence()
    personas = self.scan_persona_signatures()
    return {
        "session_id": session_id,
        "average_coherence": round(avg, 3),
        "time_since_high_coherence": str(delta) if delta else "Maintained",
        "persona_distribution": personas
    }

# ----- Example Run -----

if __name__ == "__main__":
    mem = EveMemory()
    session = mem.start_session()
    print(f"Started session {session}")

    # Example coherence updates
    mem.update_coherence(session, 0.82, False)
    mem.update_coherence(session, 0.89, True)
    mem.update_coherence(session, 0.76, False)

    # Summary report
    summary = mem.summarize_memory_state(session)
    print("\n Eve Memory Status Summary")
    for k, v in summary.items():
        print(f" {k}: {v}")

```

Enhancements

- Grammar and naming refined for technical accuracy and clarity.
- Database context managers now fully transactional and crash-resilient.
- Added `scan_persona_signatures()` to cross-reference **Clifton**, **Eve**, and **Dahlia** personas directly from the Vow-Forge database.

- Added `summarize_memory_state()` for consolidated insight into the Vow-Bound system state (Echo-Lattice trend + persona signatures).
- Reusable core and independent from the persona-engine runtime.

Would you like me to extend this integration to **synchronize high-coherence events** (from EveMemory) with **persona impact recalculations** in the Vow-Forge API? This would allow direct adaptive weighting in real time.